

Lecture-07 LOOP

A loop is a software program or script that repeats the same instructions or processes the same information over and over until receiving the order to stop.

A loop consists of two parts, a body of a loop and a control statement. The control statement is a combination of some conditions that direct the body of the loop to execute until the specified condition becomes false. The purpose of the loop is to repeat the same code a number of times.

Types of Loops in C

Depending upon the position of a control statement in a program, looping in C is classified into two types:

1. Entry controlled loop
2. Exit controlled loop

In an **entry controlled loop**, a condition is checked before executing the body of a loop. It is also called as a pre-checking loop.

In an **exit controlled loop**, a condition is checked after executing the body of a loop. It is also called as a post-checking loop.

While Loop in C

A while loop is the most straightforward looping structure. Syntax of while loop in C programming language is as follows:

```
while (condition)
{
    statements;
}
```

It is an entry-controlled loop. In while loop, a condition is evaluated before processing a body of the loop. If a condition is true then and only then the body of a loop is executed. After the body of a loop is executed then control again goes

back at the beginning, and the condition is checked if it is true, the same process is executed until the condition becomes false. Once the condition becomes false, the control goes out of the loop.

Example :

```
#include<stdio.h>
int main()
{
    int num=1;    //initializing the variable
    while(num<=10)    //while loop with condition
    {
        printf("%d ",num);
        num++;    //incrementing operation
    }
    return 0;
}
```

Output :

1 2 3 4 5 6 7 8 9 10

Description

1. We have initialized a variable called num with value 1. We are going to print from 1 to 10 hence the variable is initialized with value 1. If you want to print from 0, then assign the value 0 during initialization.
2. In a while loop, we have provided a condition (num<=10), which means the loop will execute the body until the value of num becomes 10. After that, the loop will be terminated, and control will fall outside the loop.
3. In the body of a loop, we have a print function to print our number and an increment operation to increment the value per execution of a loop. An initial value of num is 1, after the execution, it will become 2, and during the next execution, it will become 3. This process will continue until the value becomes 10 and then it will print the series on console and terminate the loop.

Do-While loop in C

A do...while loop in C is similar to the while loop except that the condition is always executed after the body of a loop. It is also called an exit-controlled loop.

Syntax of do...while loop in C programming language is as follows:

```
do
{
    statements
}
while (expression);
```

As we saw in a while loop, the body is executed if and only if the condition is true. In some cases, we have to execute a body of the loop at least once even if the condition is false. This type of operation can be achieved by using a do-while loop.

In the do-while loop, the body of a loop is always executed at least once. After the body is executed, then it checks the condition. If the condition is true, then it will again execute the body of a loop otherwise control is transferred out of the loop.

Similar to the while loop, once the control goes out of the loop the statements which are immediately after the loop is executed.

The critical difference between the while and do-while loop is that in while loop the while is written at the beginning. In do-while loop, the while condition is written at the end and terminates with a semi-colon (;)

Example:

```
#include<stdio.h>
int main()
{
    int num=1;    //initializing the variable
    do           //do-while loop
    {
        printf("%d ",2*num);
        num++;    //incrementing operation
    }
```

```
while(num<=10);  
    return 0;  
}
```

Output:

1 2 3 4 5 6 7 8 9 10

Description :

1. First, we have initialized a variable 'num' with value 1. Then we have written a do-while loop.
2. In a loop, we have a print function that will print the series by multiplying the value of num with 2.
3. After each increment, the value of num will increase by 1, and it will be printed on the screen.
4. Initially, the value of num is 1. In a body of a loop, the print function will be executed in this way: $2 * \text{num}$ where $\text{num}=1$, then $2 * 1 = 2$ hence the value two will be printed. This will go on until the value of num becomes 10. After that loop will be terminated and a statement which is immediately after the loop will be executed. In this case return 0.

For loop in C

A for loop is a more efficient loop structure in 'C' programming. The general structure of for loop syntax in C is as follows:

```
for (initial value; condition; incrementation or decrementation )  
{  
    statements;  
}
```

- The initial value of the for loop is performed only once.

- The condition is a Boolean expression that tests and compares the counter to a fixed value after each iteration, stopping the for loop when false is returned.
- The incrementation/decrementation increases (or decreases) the counter by a set value.

Example:

```
#include<stdio.h>
int main()
{
    int number;
    for(number=1;number<=10;number++) //for loop to print 1-10 numbers
    {
        printf("%d\n",number);        //to print the number
    }
    return 0;
}
```

Output:

1 2 3 4 5 6 7 8 9 10

Description :

1. We have declared a variable of an int data type to store values.
2. In for loop, in the initialization part, we have assigned value 1 to the variable number. In the condition part, we have specified our condition and then the increment part.
3. In the body of a loop, we have a print function to print the numbers on a new line in the console. We have the value one stored in number, after the first iteration the value will be incremented, and it will become 2. Now the variable number has the value 2. The condition will be rechecked and since the condition is true loop will be executed, and it will print two on the screen. This loop will keep on executing until the value of the variable becomes 10. After that, the loop will be terminated, and a series of 1-10 will be printed on the screen.

Jump Statements

(Break, Continue and Go To)

Break

The break statement is used inside loop or switch statement. When compiler finds the break statement inside a loop, compiler will abort/terminate the loop and continue to execute statements followed by loop.

Example:

```
#include<stdio.h>
int main()
{
    int x;
    scanf("%d",&x);
    for(int i=0;i<=x;i++)
    {
        if(i==3)
            break;
        printf("%d ",i);
    }
}
```

Output : 6

0 1 2

Continue Statement

The continue statement is also used inside loop. When compiler finds the continue statement inside a loop, compiler will not skip all the following statements in the loop and resume the loop.

Example:

```
#include<stdio.h>
int main()
{
    int x;
    scanf("%d",&x);
    for(int i=0;i<=x;i++)
    {
        if(i==3)
            continue;
        printf("%d ",i);
    }
}
```

Output: 6

0 1 2 4 5 6

GOTO Statements

The goto statement is a jump statement which jumps from one point to another point within a function.

```
#include<stdio.h>
int main()
{
    printf("\nStatement 1.");
    printf("\nStatement 2.");
    printf("\nStatement 3.");
    goto last;
    printf("\nStatement 4.");
    printf("\nStatement 5.");
    last:printf("\nEnd of Program." );
}
```

```
#include <stdio.h>
int main()
{
    int sum=0;
    for(int i = 0; i<=10; i++){
        sum = sum+i;
        if(i==5){
            goto addition;
        }
    }

    addition:
```

	<pre>printf("%d", sum); return 0; }</pre>
Output: Statement 1. Statement 2. Statement 3. End of Program.	Output : 15